Polynomials

In this assignment you implement a library for doing computation on polynomials in x with rational coefficients, such as $3x^2 - x + \frac{1}{2}$. Because the number of terms in a polynomial changes as we run operations on it, we use **lists** to represent the polynomial.

There are two common ways to represent polynomials using a list:

- **dense**: store one entry per power of x, zeros included; the exponent of each term is determined by the position. $3x^4 + 1$ is `[1, 0, 0, 0, 3]` (so the n-th term is at position n of the list).
- **sparse**: one entry per nonzero term. $3x^4 + 1$ is `[3x^4, 1x^0]`, but we need to track the degree of each term.

The scaffold compiles as given, but tests fail until you fill in the `TODO`s. You should submit the whole lab folder, so it's easier to test on our end. Also submit short written answers to the questions at the end of this document, a paragraph each. One pdf or plain text file, submitted alongside your code.

Checklist

For each section, that functionality is tested by the test section in brackets.

Storage and iteration, in both `DensePolynomial` and `SparsePolynomial` [`construct`]

- ☐ Constructor: create the empty list, `ArrayList` or `LinkedList`. The rest of the constructor is given.
- ☐ `addTerm(c, e)`: `this += c x^e`, keeping the invariant (see *Representations*).
- ☐ `terms()`: iterator over the nonzero terms, highest exponent first. For dense, an inner class.
- ☐ `degree()`: highest exponent, -1 for the zero polynomial.

Written once, in `Polynomial`

- ☐ `equals`, to check if two polynomials are the same (even though they may be represented differently!) [`equals`]
- ☐ `evaluate(x)`: the value of the polynomial at `x` [`evaluate`]

In-place operations, in both subclasses

- ☐ `addInPlace(other)`: `this += other`. [`addInPlace`, `add`, `subtract`]
- ☐ `multiplyInPlace(c, e)`: `this *= c x^e`. [`multiplyInPlace`, `negate`, `multiply`, `divide`]
- ☐ `derivativeInPlace()` [`derivativeInPlace`, `derivative`]

Long division, written once in `Polynomial` [`divide`]

- ☐ `divide(b)`: the quotient and the remainder of `this` divided by `b`, as a `DivisionResult` (see *Long division*).

Questions

- ☐ Short answers to the questions at the end of this document, a paragraph each, in a pdf or plain text file.

Optional tasks

These are not needed to pass the tests. Each one is about speed: check that the tests still pass, then measure the difference with `Benchmark`.

- ☐ The dense fast route in `addInPlace` (see *The dense fast route*). Measure with `java Benchmark add 20000`. [`addInPlace`, `add`]
- ☐ Faster `evaluate`: Horner's method, and its variation for sparse polynomials (see *Written once in `Polynomial`*). Measure with `java Benchmark evaluate 20000`. [`evaluate`]
- ☐ Override `subtractInPlace` in one or both subclasses with a single-pass version (see *`subtractInPlace`*). Measure with `java Benchmark subtract 20000`, which also times the given version. [`subtract`, `addInPlace`]
- ☐ Switch the `List` of one or both subclasses to the other implementation (see *Which `List`*), and compare the benchmarks of every operation before and after.

Rules

- `addTerm` and the methods ending in `InPlace` change `this`. Every other operation returns a **new** polynomial of the **same class as `this`**, and leaves its operands unchanged.

- `other` may be of either class. Read it only through `other.terms()` and `other.degree()`, except in the dense fast route (see below).
- Each subclass stores the polynomial only in its `List` field: no arrays, no other collections.
- You may add private methods and fields. Don't change anything else in the given files.

Reference

Given classes and their methods

`Rational`: immutable fraction, always in lowest terms.

- `a.add(b)`, `a.negate()`, `a.multiply(b)`, `a.divide(b)` return new objects.
- `a.pow(k)`: a^k for $k \geq 0$.
- `isZero()`, `isNegative()`, `isInteger()`.
- `new Rational(3)`, `new Rational(3, 4)`, `Rational.ZERO`, `Rational.ONE`.
- Compare with `a.equals(b)`, never `a == b`.

`Term`: one term $c\,x^e$, immutable. `getCoef()` (a `Rational`), `getExp()` (an `int`).

`DivisionResult`: what `divide` returns, immutable. `getQuotient()`, `getRemainder()`.

`Parser.parse("3x^2 - x + 1/2")`: a `List<Term>`, in the order of the text without summing them, so `"x + x + 0"` gives three terms.

`Polynomial`

What each subclass implements:

```
public abstract Iterator<Term> terms();              // nonzero terms, highest exponent first
public abstract int degree();                        // -1 for the zero polynomial
public abstract void addTerm(Rational c, int e);     // this += c x^e
public abstract void addInPlace(Polynomial other);   // this += other
public abstract void multiplyInPlace(Rational c, int e);   // this *= c x^e
public abstract void derivativeInPlace();            // this = this'
protected abstract Polynomial newZero();             // given: empty polynomial of the same class
```

`terms()` returns a new iterator on every call, over the nonzero terms only, each exponent at most once.

Everything else is given, built on these, except `evaluate`, `equals` and `divide`: `copy`, `add`, `subtract`, `subtractInPlace`, `negate`, `multiply`, `derivative`, `toString`, `hashCode`, `isZero()`, `leadingTerm()` (the term with the highest exponent), `termCount()`, `density()` (fraction of nonzero coefficients). For example:

```
public final Polynomial add(Polynomial other) {
    Polynomial result = copy();
    result.addInPlace(other);
    return result;
}
```

`multiply` adds up, with `addInPlace`, a copy of `this` multiplied in place by each term of `other`.

The constructors call `addTerm` once per input term, so any input (any order, repeated exponents, zeros) ends up normalized.

Representations

Remember that "invariant" is a property of a class that must be preserved after every operation that changes the class. (The invariant can be violated temporarily inside the method itself, but the end result should *always* respect the invariant.)

- **Dense:** `List<Rational> coefficients`, where `get(i)` is the coefficient of x^i. **Invariant**: the last element is nonzero.

- **Sparse:** `List<Term> termList`. **Invariant**: sorted by exponent (ascending or descending, your choice), only one term per exponent, no zero coefficients.

In both, the zero polynomial is the empty list.

`addTerm`.

- Dense: grow the list with zeros up to index `e` if needed; afterwards, remove trailing zeros.
- Sparse: find where exponent `e` belongs; combine with the term already there (remove it if the sum is zero), or insert a new term. A `ListIterator` does all three.

`addInPlace`. The most general route is one `addTerm` per term of the `other` polynomial. It is correct in both classes, but slower than necessary:

- Sparse: both lists are sorted, so do a single pass over the two (the merge step of merge sort), inserting the terms of `other` into `termList` with one `ListIterator`. Once the terms of `other` run out, the pass stops.
- Dense: one `addTerm` per term is already linear, since `addTerm` reaches index `e` directly. When `other` is dense too, the optional fast route below saves the remaining work.

`other` may be `this`: doing `p.addInPlace(p)` introduces a problem called *aliasing*. The scaffold already handles it, by replacing `other` with a copy at the start, so you never iterate over the list as you are changing it.

`multiplyInPlace`. Multiplying by $c\,x^e$ multiplies every coefficient by c and adds e to every exponent. Dense: the coefficients move e positions up. If $c = 0$, the result is the zero polynomial.

`derivativeInPlace`. $c\,x^e$ becomes $ec\,x^{e-1}$; the constant term disappears, and degree goes down by 1.

Dense `terms()`: an inner class `implements Iterator<Term>`, as for `MyArrayListDouble`, walking the list from the highest index down and skipping zeros.

The dense fast route (optional). Through `terms()`, a dense `other` creates a new `Term` object per coefficient, only to put each coefficient back at its index. When both operands are dense, `addInPlace` can be implemented with a loop over two lists side by side. The scaffold already has the check; if you fill it in, end it with `return`, so the general route below it is skipped:

```
if (other instanceof DensePolynomial d) {
    // fast route: d.coefficients is the other list
}
// general route, through other.terms()
```

`d.coefficients` compiles because `private` fields are accessible from any object of the same class. Sparse needs no fast route: its `terms()` already walks its list.

Which `List`. In-place operations insert, remove and replace elements while walking the list. Think about what each costs in an `ArrayList` and in a `LinkedList`, and about `get(i)` in a loop, before choosing. Switching is one line in the constructor; the benchmark shows the difference.

Written once in `Polynomial`

`evaluate(x)`: the value of the polynomial at `x`, using `terms()`. Optionally, you can make it faster for dense polynomials by implementing Horner's method (look it up). However this doesn't work well for sparse polynomials: you can make it faster on sparse polynomials too by implementing a variation of Horner's method that evaluates each "gap" of `k` zero coefficients as one `x.pow(k)`.

`equals(o)`: true if `o` is a `Polynomial` with the same terms, whatever its class. Walk both `terms()` side by side.

Long division

Dividing a by b gives a quotient q and a remainder r such that $a = q \cdot b + r$ and $\deg r < \deg b$. It works like long division of numbers, one term of the quotient at a time:

1. Start with $q = 0$ and $r = a$.
2. While $\deg r \geq \deg b$:
   a. Divide the leading term of r by the leading term of b: this is the next term of the quotient, $m\,x^k$, with m the ratio of the two coefficients and k the difference of the two exponents. Add it to q.

   b. Subtract $m\,x^k \cdot b$ from r. This cancels the leading term of r, so its degree goes down.
3. q is the quotient, r the remainder.

Example: $(2x^3 + 3x^2 + 1) \div (x + 1)$.

- $r = 2x^3 + 3x^2 + 1$: next term $2x^3/x = 2x^2$; subtract $2x^2(x + 1) = 2x^3 + 2x^2$; now $r = x^2 + 1$.
- $r = x^2 + 1$: next term $x^2/x = x$; subtract $x(x + 1) = x^2 + x$; now $r = -x + 1$.
- $r = -x + 1$: next term $-x/x = -1$; subtract $-1(x + 1) = -x - 1$; now $r = 2$.

$\deg 2 = 0 < 1$, so it stops: $q = 2x^2 + x - 1$, $r = 2$. Check: $(2x^2 + x - 1)(x + 1) + 2 = 2x^3 + 3x^2 + 1$.

Coefficients are rational, so every division of coefficients is exact: $(x^2 + 1) \div 2x$ gives $q = \frac{1}{2}x$, $r = 1$.

With the methods you have:

- q and r: `newZero()` and `copy()`, so both are of the same class as `this`.
- Leading terms: `leadingTerm()` (given), then `getCoef()` and `getExp()`.
- Adding $m\,x^k$ to q: `addTerm`.
- Subtracting $m\,x^k \cdot b$ from r: copy b, `multiplyInPlace` the copy by $-m\,x^k$, `addInPlace` it into r.
- Dividing by the zero polynomial throws `ArithmeticException`.

Return both with `new DivisionResult(q, r)`.

`subtractInPlace`

`subtractInPlace` is given: it copies `other`, negates the copy with `multiplyInPlace`, and adds it with `addInPlace`. Overriding it with a single pass is optional. If you do, start it with the same line as `addInPlace`, `if (other == this) other = copy();`: in `p.subtractInPlace(p)` every term cancels and is removed while you iterate. Remember to `@Override`.

Tests

`Tests` checks results through `terms()` and `degree()` only. Every operation with two operands runs on all four combinations of classes; in-place tests check that `other` is unchanged, the others that both operands are. Once `terms()` works, so does the given `toString`, useful for debugging. A test running longer than 5 seconds fails, and the rest of its section is skipped.

Benchmark

```
java Benchmark multiply 1000 100    degree 1000, 100 nonzero terms
java Benchmark multiply 1000        degree 1000, from 1 nonzero term up to all 1001
```

Times one operation (`add`, `subtract`, `multiply`, `derivative`, `evaluate`) on random dense and sparse polynomials. Suggested degrees: 20000, and 1000 for `multiply`. A column slower than 5 seconds stops being timed.

Questions

Answer these in your submitted file, a paragraph each.

1. Which polynomials fit the dense representation best, and which fit the sparse one best?
2. Which `List` did you use in each subclass, and why? If you tried both, what changes did you notice, if any?
3. Does your implementation of `evaluate` get faster when the polynomial is sparse? Why or why not?
4. Assuming each `Rational` operation is 1 operation, describe the worst-case complexity of your implementation of `addInPlace`, in the cases where `this` and `other` are both dense or both sparse: state their big-O complexity in terms of t_0, d_0 (number of terms and degree of `this`) and t_1, d_1 (the same for `other`).